

Get It — Technical Writeup

"What I cannot create, I do not understand." — Richard Feynman, last blackboard at Caltech, February 15, 1988.

Built at GDG AI Hack 2026, Milan — for the Braynr challenge.

By Mattia Beltrami (Politecnico di Milano), Matteo Impieri (Politecnico di Milano), Filippo Difronzo (Politecnico di Milano), Luca Feggi (Università di Padova).

What we built and why it exists

The student already has the PDF. They don't need another summary; they need a way to see the parts of the document that text alone refuses to explain, and they need a way to *prove to themselves* that they have understood — concept by concept, not page by page. Today that proof is missing: ratings on flashcards measure recall in the moment, mindmaps measure how much you drew, summaries measure how patient the AI was. None of these measure whether the student would survive a question they hadn't seen before.

Get It is built around two convictions. First, that **the document stays at the center**: every visualization, every chat, every flashcard, every Feynman session is grounded in the source PDF — never in general world knowledge. Second, that **mastery has structure**: it is not a single number, it is four orthogonal signals (memory, comprehension, structure, application) that move at different speeds and respond to different evidence. Get the structure right and the rest of the product writes itself.

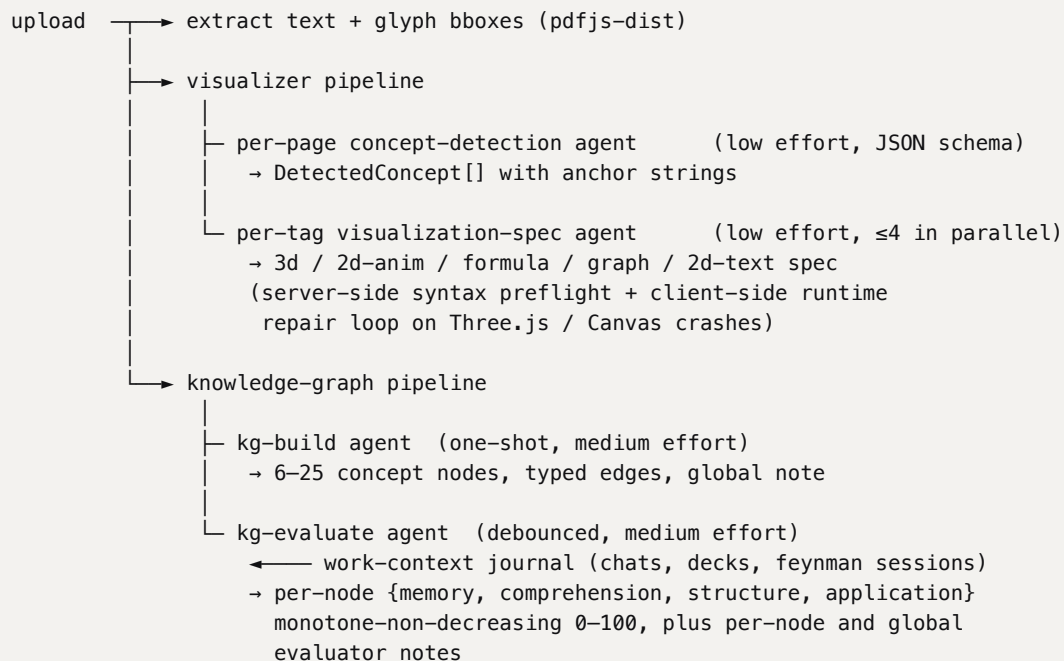
We adopted Alessandro de Concini's own paradox as our brief: the Feynman technique is the most powerful learning tool ever conceived, and it is so slow that almost no one uses it across an exam syllabus. Our answer is not to abolish the slowness — depth costs time — but to remove the friction that has nothing to do with thinking: the blank page, the missing diagram, the lack of a listener. Three tools (chat, flashcards, Feynman-with-a-curious-child) feed a single per-document journal. One evaluator agent reads that journal end-to-end and updates a knowledge graph that the student can read back as a map of where they actually are.

The macro user loop is therefore a closed cycle: the visualizer generates time-to-value the moment a PDF lands; the three tools generate study evidence; the evaluator turns evidence into a four-axis score per concept; the knowledge graph back-reflects that score onto the document and onto the next-best action. Output becomes outcome.

System architecture

The product is a single Next.js 16 application (App Router, React 19, server components for the entry points, client components for the orchestrators) deployed locally against the user's own Codex CLI. There is no proprietary model in the loop — every agent call is a `codex exec` invocation through the official `@openai/codex-sdk`, constrained by a strict per-call JSON Schema so a "concept" or a "knowledge-graph node" is always a typed object, never free text the UI has to parse defensively.

Two macro-pipelines run **in parallel from the moment the PDF is uploaded**:



The visualizer pipeline is the one Braynr scores on time-to-value: tags appear inline in the document the instant detection returns; the right pane fills in by itself as visualizations land. Up to four generations run in parallel client-side; the orchestrator keeps a queue capped at `MAX_CONCURRENT_VIZ_GEN`, an in-flight set, and a per-tag retry budget. When a visualization throws a runtime error in the sandbox we *do not* surface a stack trace — we capture the error message, hand it back to Codex with the broken `setup_code` as repair context, and the visualizer re-renders. The user sees "repairing — attempt N of M" instead of red text.

The knowledge-graph pipeline is the one that turns Get It from a viewer into a measurement instrument. It is the layer the original Braynr architecture does not have, and it is where the four-axis rubric lives. Both agents talk to the same Codex provider as the visualizer — same SDK wrapper, same schema-enforced replies — so there is one auth path, one error mode, one budget.

The four-axis evaluator

Every concept node carries four scores from 0 to 100:

AXIS	WHAT IT MEASURES	STRONGEST SIGNAL
memory	recall over time	flashcard ratings (1–4), recall references in chat, FSRS-style stability
comprehension	understanding in the student's own words	original metaphors in chat, plain-language Feynman explanations
structure	grasp of how concepts connect	multi-step reasoning that bridges concepts, references to prerequisites
application	transfer to new cases	original examples, edge cases, novel problem solving

The evaluator agent sees the entire work context (compacted, with timestamps) plus the current graph plus the previous scores. It is told, in its system prompt, that scores are **monotone non-decreasing** — the student can only progress, never regress — and the runtime enforces this with a clamp on every update so a chatty interaction can't accidentally erase prior evidence. The agent is also told that *quantity does not entitle a score*: a student who has done fifty cards and rated them all "Again" sits low on memory; a student who has given a single brilliant Feynman explanation can earn a strong jump on comprehension for that one concept. Empty work context produces no movement at all — the rubric is observable-evidence-based by construction.

Scheduling matters as much as the rubric. The evaluator is expensive (one Codex turn at medium effort per pass) and the chat tool is chatty by definition, so we run a per-doc queue with at most one in-flight pass and one pending. After every assistant chat reply, every ended flashcard deck, and every completed Feynman session, the API route fires `scheduleEvaluation(docId)` and returns immediately. The client polls `/api/kg/[docId]/state` (which exposes the live `evaluating` flag from the queue), accelerating to 2.5 s while the agent is working and slowing to 6

s when idle. The badge in the top tab bar reads "Building graph", "Evaluating", "No evaluations yet", or "Synced 12 s ago" depending on what the queue is doing.

The three study tools and the work-context journal

The three tools are deliberately small and deliberately different. **Chat** is the most familiar surface — multi-turn, multi-thread, scoped to one document, with the knowledge graph and a doc excerpt injected as system context. **Flashcards** is the active-recall engine: Codex generates a 4–10 card deck for a topic, the student types their answer (optionally) and self-grades 1–4 (Again / Hard / Good / Easy, the FSRS convention) per card; ratings are recorded card-by-card and the deck is closed once every card is graded. **Feynman** is the showpiece: the agent plays a curious eight-year-old who asks three to four short, pointed questions; the student is forced into the role of the teacher; after the last turn a separate summary call writes a 3–6-sentence honest read of where the explanation held and where it broke down. The session is bounded so the data stays usable for the evaluator and the student doesn't drift.

Behind all three tools sits a single artifact: the **work-context JSON**, one file per doc on the server, append-only by convention. Every chat message, every card rating, every Feynman turn lands here with a UTC timestamp. This is the file the evaluator reads, the file the student can download from the right-pane menu, and the file that — by design — is the only thing the system needs to remember about a study session. We give the student exactly what we use.

Implementation choices we are happy with

Persistent state is filesystem-backed under `/tmp/braynr-uploads/<docId>` (PDF + extracted text + work-context + KG). Process-local, cheap, recoverable, and a clear seam if we ever lift it to S3 or Postgres. Client state (tags, active selection, settings) is sessionStorage-backed so reload restores it without a server round-trip; we re-fire orchestration that died with the page (in-flight visualizations marked `generating: true` get re-enqueued on mount).

Types are split into `*-types.ts` modules (pure TS, no `node:fs` imports) and `*.ts` modules (the actual storage helpers). This sounds pedantic until you discover that Next.js bundles a transitively-imported module into the client when *any* type from it is referenced — even a bare `import type {}`. Splitting types into a node-free file is the only way to keep `lib/kg.ts` and `lib/work-context.ts` server-only without poisoning the browser bundle. We learned that the hard way and the comments in those files say so.

The visualizer sandbox runs LLM-emitted JavaScript inside a `new Function` IIFE with all dangerous globals (`window`, `document`, `fetch`, `XMLHttpRequest`, `WebSocket`, `Function`, `eval`, `localStorage`, `sessionStorage`, `require`) shadowed as `undefined` parameters. It is a defense against LLM mistakes, not a defense against adversarial input — the user runs their own Codex account against their own PDFs. The boundary is reasonable for the demo and explicitly documented.

Settings (auto-generate visualizations, max viz repair attempts) are runtime-mutable from a popover in the top tab bar. Both default to the env values from `lib/config.ts` (`NEXT_PUBLIC_AUTO_GENERATE_VIZ`, `NEXT_PUBLIC_MAX_VIZ_GEN_RETRIES`) and persist to sessionStorage. Toggling auto-generate from off to on sweeps already-detected idle tags into the queue immediately; on to off lets in-flight calls finish naturally.

What's out of scope and why

A vocal mode for Feynman is the natural next step — the same agent loop, the same end-of-session summary, with a streaming TTS layer over the child voice. We built the text variant for the hackathon because typed transcripts are strictly better evaluator inputs (no transcription error, no per-token cost, full searchability) and because the role-reversal UX — *you* are teaching, *the agent* is the audience — is what changes the affect, not the modality. Audio is the obvious extension; the data shape doesn't change.

A multi-user backend, real auth, and a hosted deployment are also deliberately out of scope. Get It runs locally against the user's own Codex login, against their own PDFs, on their own machine, by design. The Braynr policy band — *source-grounded only, local-first, tiered access* — is something we get for free at this scale; we did not have to architect away from it.

What we want a judge to remember

Three things. (1) The document is the center, and every other surface back-reflects to it — the visualizer pulls from the page text, the chat injects the page text, the evaluator reads the journal *and* the page text. (2) Mastery is four numbers and they only ever go up — that one constraint is the difference between a study app and a measurement instrument. (3) The same Codex provider drives every agent — concept detection, visualization generation, knowledge-graph build, evaluator, chat, flashcards, Feynman child, Feynman summary — eight prompts behind one auth path, eight schemas behind one shared SDK wrapper. The system is a sum of small, schema-typed turns, not a god-prompt; that is what makes it debuggable and that is what will let it grow.

|"Their knowledge is so fragile." — Feynman, 1985. We took the line literally.